Solana

Integrating with @solana/kit

Open in Claude

Environment

React

Privy's **ConnectedStandardSolanaWallet** object is fully compatible with popular web3 libraries for interfacing wallets and signing transactions and messages, such as `@solana/kit`.

Read below to learn how to best integrate Privy alongside `@solana/kit`.

First find your desired wallet from the `wallets` array:

```
import {useWallets} from '@privy-io/react-auth/solana';

const {wallets} = useWallets();
const wallet = wallets[0]; // Replace this with your desired wallet
```

Signing Transactions

Transactions generated by `@solana/kit` can be signed using the `signTransaction` method from the `useStandardSignTransaction` hook.

privy

```
import {
  pipe,
  createTransactionMessage,
  setTransactionMessageFeePayer,
  setTransactionMessageLifetimeUsingBlockhash,
  appendTransactionMessageInstructions,
  compileTransaction,
  createNoopSigner,
  createSolanaRpc,
  getTransactionEncoder
} from '@solana/kit';
import {getTransferSolInstruction} from '@solana-program/system';
import {useStandardSignTransaction} from '@privy-io/react-auth/solana';

const {signTransaction} = useStandardSignTransaction();

const LAMPORTS_PER_SOL = 1_000_000_000;

const transferInstruction = getTransferSolInstruction({
  amount: LAMPORTS_PER_SOL * 1,
  destination: address(to),
  source: createNoopSigner(address(wallet.address))
});

const {getLatestBlockhash} = createSolanaRpc('YOUR_SOLANA_RPC_URL');
const {value: latestBlockhash} = await getLatestBlockhash().send();

// Create transaction
const transaction = pipe(
  createTransactionMessage({version: 0}),
  (tx) => setTransactionMessageFeePayer(address(wallet.address), tx),
  (tx) => setTransactionMessageLifetimeUsingBlockhash(latestBlockhash, tx),
  (tx) => appendTransactionMessageInstructions([transferInstruction], tx),
  (tx) => compileTransaction(tx)
);
const encodedTransaction = getTransactionEncoder().encode(transaction);

// Sign the transaction
const signedTransaction = await signTransaction({
  transaction: new Uint8Array(encodedTransaction),
```

```
  wallet: wallet
});
```

Sending Transactions

Transactions signed using the `signTransaction` method can be sent using the `signAndSendTransaction` method from the `useStandardSignAndSendTransaction` hook.

```
import {useStandardSignAndSendTransaction} from '@privy-io/react-au

const {signAndSendTransaction} = useStandardSignAndSendTransaction();

const signature = await signAndSendTransaction({
  transaction: new Uint8Array(transaction), // The transaction to send, from the
  wallet: wallet
}).signature;
```

Was this page helpful? Yes No

Integrating with @solana/web3.js

Sign transaction (TEE mode)

Powered by mintlify